

Class imbalance methods for multi-label classification (literature reference)

Source: Sophie Henning, William Beluch, Alexander Fraser, Annemarie Friedrich. “A Survey of Methods for Addressing Class Imbalance in Deep-Learning Based Natural Language Processing.” EACL 2023 (CORE rank A). <https://aclanthology.org/2023.eacl-main.38/>

Filtered to only the methods the paper confirms as applicable to **multi-label** classification (our setting – CAM-LDS samples can have multiple true tactics at once, e.g. TA0011,TA0001,TA0003,TA0004,TA0005 for a single sample). Resampling methods (ROS/RUS, CAS) and GE3/MISO/WCE/RL were excluded – the paper marks these “?” (untested) or “N/A”/“x” (not applicable) for multi-label.

Category	Method	What it is	Code
Data Augmentation	EDA	“uses dictionary-based synonym replacements, random insertion, random swap, and random deletion”	Y
	TextCut	“randomly replaces small parts of the BERT representation of one instance with those of the other”	N

Category	Method	What it is	Code
Loss Functions	ECRT	“learns to map encoder representations... to a new space... whose components are independent of each other given the class,” enabling generation of “new meaningful minority examples by permuting or sampling components in the source space”	Y
	FL (Focal Loss)	“down-weights instances for which the model is already confident (implemented with the $(1 - p_j)^{\beta}$ coefficient)”	Y
	ADL	ADL extends Dice loss, which captures class-wise F1, with confidence-based down-weighting of easy predictions.	Y
	LDAM	LDAM uses larger class-dependent margins for minority classes to improve their separation and generalization.	Y

Category	Method	What it is	Code
Staged Learning	WBCE	WBCE assigns higher weights to minority classes to mitigate class imbalance, but also increases the weight of their negative instances.	N/A
	DB	adds “Negative Tolerant Regularization for the loss for negative classes... imposes a sharp drop in the loss function for negative classes once the respective logit is below a threshold”	Y
	cRT	“two-stage classifier re-training... using the original distribution in the first stage... employs [class-balanced sampling] only in the second stage after freezing the representation weights”	Y
	ST (Sequential Targeting)	“model[s] imbalanced classification as a continual learning task with k stages where the data gradually becomes more balanced”	Y

Category	Method	What it is	Code
Model Design	tau-norm	“normalize the classifier weights directly in one-staged training using a hyperparameter tau to control the normalization ‘temperature’ ”	Y
	SetConv	Learns class representatives from support sets using convolution kernels to capture intra- and inter-class correlations.	N
	ProtoBERT	Uses class centroids in a learned BERT feature space to classify instances based on similarity to class representatives.	Y
	HSCNN	“uses class representatives only for the classification of tail classes, while head classes are assigned using a standard text CNN... assigns a tail class if the similarity to the class representative... exceeds 0.5”	N

Notes for our project

- We have already tried the **Loss Functions** family (uniform loss = label-blind spreading, class-reweighting = WBCE-style) – see

tactic_expansion_comparison.md. WBCE’s own known weakness (upweighting a class also upweights its negative instances, discouraging predicting it) matches what we saw with uncapped reweighting overcorrecting.

- **DB (Distribution-Balanced Loss)** is untested by us and directly targets the “logit suppression” mechanism – caps how much a class gets pushed away once the model is already confident it’s negative, rather than letting the punishment build up indefinitely. Confirmed large multi-label gains in the source paper (Huang et al. 2021).
- **cRT / ST (Staged Learning)** is untested by us – train normally first, then freeze/adjust in a second stage. Matches the “freeze the dominant persistence/privilege_escalation prototypes, keep training the rest” idea discussed as a next step.
- **ProtoBERT** is the closest architectural relative to what we’re already doing (prototype/class-centroid based) – worth reading in more detail since it’s the same family as our `multilabel_prototype_loss`.